Reviewer Pack — Minimal Rank of Geometric Langlands Sheaves Bounds BP Read-T…

Entry c17dbbc1f0ee · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 14:10:11 UTC

Minimal Rank of Geometric Langlands Sheaves Bounds BP Read-Twice Size

Entry ID: `c17dbbc1f0ee` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 14:10:11 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Program (Sheaves) **Field B** (complexity object): Branching program: read-twice (BP)

Statement:

> {'text': 'For every n-variable 3-CNF formula F, the minimal rank of the sheaf associated with the corresponding geometric Langlands space is proportional to the size of a minimal read-twice BP for F, i.e., ρ_sheaf(F) = Θ(size(BP_readtwice(F)))', 'counterexample': 'A counterexample can be constructed by finding a 3-CNF formula F for which the associated geometric Langlands sheaf has a low rank but requires an exponentially large read-twice BP to be refuted.'}

Rationale (proposer's reasoning):

> {'text': 'The Geometric Langlands Program provides a framework that links algebraic geometry with mathematical physics. By studying the ranks of sheaves in this program, one may uncover new structural properties related to computational complexity. The conjecture proposes a novel connection between geometric Langlands and branching programs, which could potentially lead to new insights into complexity theory.'}

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `aa2fb3e3c5ed31d2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for all n ≤ 40, the ratio of the minimal sheaf rank to the size of the minimal read-twice BP for each 3-CNF formula is within a threshold of ±30%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric Langlands program (sheaves)" AND "read-twice branching program" - "minimal rank of geometric Langlands sheaves" AND "size of minimal read-twice BP" - "branching program: read-twice" IN title AND "geometric Langlands space"

Top relevant hits considered: - [http://arxiv.org/abs/2503.14756v3] SceneEval: Evaluating Semantic Coherence in Text-Conditioned 3D Indoor Scene Synthesis - [http://arxiv.org/abs/2408.02211v2] SceneMotifCoder: Example-driven Visual Program Learning for Generating 3D Object Arrangements - [http://arxiv.org/abs/2509.13291v1] Towards an Embodied Composition Framework for Organizing Immersive Computational Notebooks

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(n * 3):
            clause = [random.randint(1, n), -random.randint(1, n)]
            if random.choice([True, False]):
                clause[0] *= -1
            if random.choice([True, False]):
                clause[1] *= -1
            cnf.append(clause)
        return cnf

    def backtrack():
        assignment = [None] * (n + 1)
        stack = []

        def search(t):
            if t > n:
```

```
            return True
        for val in [-1, 1]:
            assignment[t] = val
            if is_satisfiable(cnf):
                stack.append((t, val))
                if search(t + 1):
                    return True
                stack.pop()
        assignment[t] = None
        return False

    return search(1)

def is_satisfiable(cnf):
    while stack:
        t, val = stack.pop()
        assignment[t] = val
        for clause in cnf:
            if not any([assignment[abs(lit)] == (lit > 0) for lit in clause]):
                break
        else:
            return True
    return False

def read_twice_bp_size(cnf):
    n = len(cnf)
    dp = [[False] * (n + 1) for _ in range(n + 1)]

    def backtrack(i, j):
        if i > n or j > n:
            return 0
        if dp[i][j]:
            return 0
        dp[i][j] = True
        size = 1
        for clause in cnf:
            if not any([assignment[abs(lit)] == (lit > 0) for lit in clause]):
                break
        else:
            size += backtrack(i + 1, j)
        return size

    return backtrack(1, 1)

n = random.randint(5, 40)
cnf = generate_cnf(n)
if not backtrack():
    return {
        "metric_name": "rho_sheaf",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "The formula is unsatisfiable."
    }

bp_size = read_twice_bp_size(cnf)
```

```
    if bp_size == 0:
        return {
            "metric_name": "rho_sheaf",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "The formula is unsatisfiable."
        }

    rho_sheaf = Fraction(n, bp_size)
    return {
        "metric_name": "rho_sheaf",
        "metric_value": float(rho_sheaf),
        "instances_tested": 1,
        "conjecture_holds": abs(rho_sheaf - 1) <= 0.3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results if r["metric_value"]
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["metric_value"] is not None for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='low rho_sheaf' first_failing_seed={first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d286dca8.py", line 118, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d286dca8.py", line 84, in run_trial
    if not backtrack():
           ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d286dca8.py", line 49, in backtrack
```

```
    return search(1)
           ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d286dca8.py", line 41, in search
    if is_satisfiable(cnf):
       ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d286dca8.py", line 52, in is_satisfiabl
    while stack:
          ^^^^^
NameError: name 'stack' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure the validity of the results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10468
2	preregistration	ollama_remote	glm4:latest	0	0	5512
3	novelty	ollama_remote	glm4:latest	0	0	4802
4	novelty	ollama_remote	glm4:latest	0	0	5833
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22004
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8753
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9466
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12068
9	judge	ollama_remote	glm4:latest	0	0	9809

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 88714 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c17dbbc1f0ee.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c17dbbc1f0ee.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c17dbbc1f0ee.tar.gz` (if generated)